

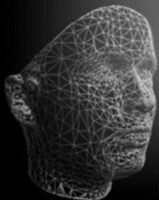
Advanced Topics in Computer Graphics II

Geometry Processing

Mesh Editing

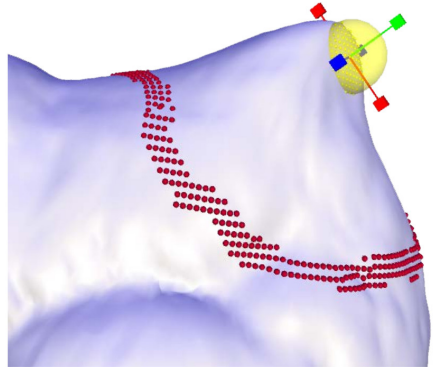


December 4, 2024



Motivation: What is Mesh Editing?

1. Start with a triangle mesh $\mathcal{M}$
2. Determine a submesh $\mathcal{R} \subset \mathcal{M}$ called the **Region of Interest (ROI)**
3. Determine **handles** within the ROI
4. Transform handles to the new pose
→ **Constraints**

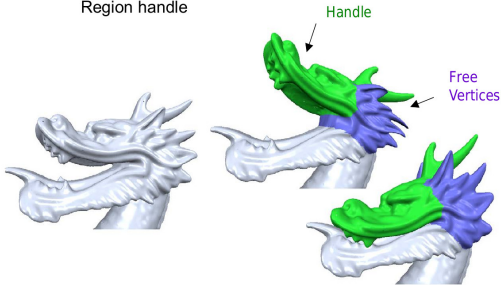


→ The algorithm moves all vertices in the ROI such that they satisfy the pose constraints and certain smoothness constraints are fulfilled

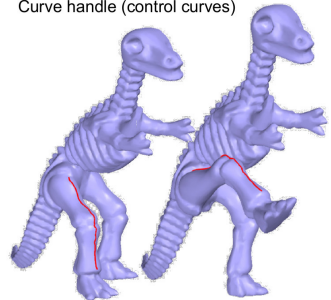


More examples:

Region handle



Curve handle (control curves)

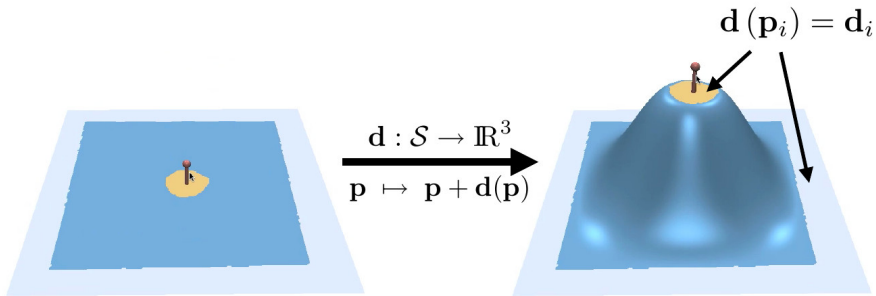




Idea: Model the mesh deformation by a displacement function¹

$$\mathbf{d}: \Omega \subset \mathbb{R}^2 \rightarrow \mathbb{R}^3$$

and constrain the displacement at the handle and the fixed regions.



¹O. Sorkine and M. Botsch. "Interactive Shape Modeling and Deformation". In: *Eurographics 2009 - Tutorials*. Ed. by K. Museth and D. Weiskopf. The Eurographics Association, 2009.



We want an intuitive deformation, so define a physically-based energy²

$$E = \int_{\Omega} k_s \left\| \mathbf{M}'_1(\mathbf{u}) - \mathbf{M}_1(\mathbf{u}) \right\|_F^2 + k_b \left\| \mathbf{M}'_2(\mathbf{u}) - \mathbf{M}_2(\mathbf{u}) \right\|_F^2 d\mathbf{u}$$

consisting of a **stretching term**, i.e. change of local distances, and a **bending term**, i.e. change of local curvature, each weighted with **stiffness terms** k_s, k_b .

Problem: The energy is non-linear and therefore hard to solve.

Idea: Linearize the energy and express it in terms of the displacement function

$$E = \int_{\Omega} k_s \left(\|\mathbf{d}_x(\mathbf{u})\|^2 + \|\mathbf{d}_y(\mathbf{u})\|^2 \right) + k_b \left(\|\mathbf{d}_{xx}(\mathbf{u})\|^2 + 2 \|\mathbf{d}_{xy}(\mathbf{u})\|^2 + \|\mathbf{d}_{yy}(\mathbf{u})\|^2 \right) d\mathbf{u}$$

where $\mathbf{u} = (u_x, u_y)^T$ is a 2D coordinate and $\mathbf{d}_x = \frac{\partial}{\partial u_x} \mathbf{d}$ and $\mathbf{d}_y = \frac{\partial}{\partial u_y} \mathbf{d}$ the first partial derivatives. The second partial derivatives are defined in the same way.

²D. Terzopoulos et al. "Elastically deformable models". In: *ACM Siggraph Computer Graphics*. Vol. 21. 4. ACM. 1987, pp. 205–214.



$$E = \int_{\Omega} k_s (\|d_x(u)\|^2 + \|d_y(u)\|^2) + k_b (\|d_{xx}(u)\|^2 + 2 \|d_{xy}(u)\|^2 + \|d_{yy}(u)\|^2) du$$

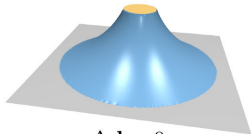
If we now apply variational calculus, we get the following Euler-Lagrange equation

$$-k_s \Delta d + k_b \Delta^2 d = 0$$

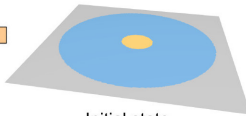
where

$$\Delta d = \operatorname{div} \nabla v = d_{xx} + d_{yy}$$

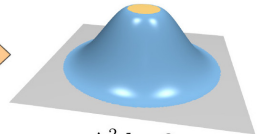
$$\Delta^2 d = \Delta(\Delta d) = d_{xxxx} + 2 d_{xxyy} + d_{yyyy}$$



$\Delta d = 0$
(Membrane)



Initial state

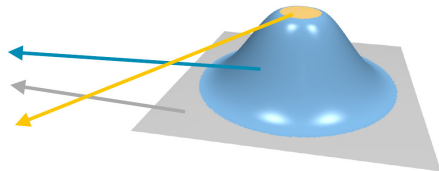


$\Delta^2 d = 0$
(Thin plate)

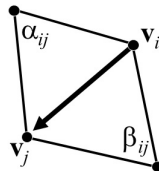
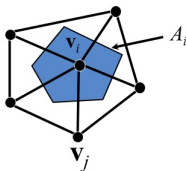
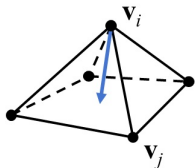


We obtain the final solution by solving a sparse linear equation system

$$\begin{pmatrix} -k_s \Delta + k_b \Delta^2 & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{I} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{I} \end{pmatrix} \begin{pmatrix} \vdots \\ d_i \\ \vdots \end{pmatrix} = \begin{pmatrix} \mathbf{0} \\ \mathbf{0} \\ d_j \end{pmatrix}$$



where the constraints for the fixed parts are needed to get a correct result.



The Laplace-Beltrami operator can be approximated using cotangent weights.



Given: A function $f: \Omega \rightarrow \mathbb{R}$ defined on the domain $\Omega \subset \mathbb{R}^2$.

Idea: Manipulate/deform the gradients³⁴ of f using a mapping T :

$$g(u) \mapsto T(g(u)) \quad , \quad g(u) = \nabla f(u)$$

Find a new function f' whose gradient field best matches the target:

$$E[f'] = \int_{\Omega} \left\| \nabla f'(u) - T(g(u)) \right\|_2^2 du$$

Applying variational calculus leads to the following Euler-Lagrange equation:

$$\Delta f'(u) = \operatorname{div} T(g(u))$$

³Y. Yu et al. "Mesh editing with poisson-based gradient field manipulation". In: *ACM Transactions on Graphics (TOG)*. vol. 23. 3. ACM. 2004, pp. 644–651.

⁴M. Botsch et al. "Deformation Transfer for Detail-Preserving Surface Editing". In: *Vision, Modeling & Visualization*. 2006.



In order to apply this method to a mesh with vertices $\{v_1, \dots, v_n\}$, we need to know how to define the vertices in terms of the scalar field f .

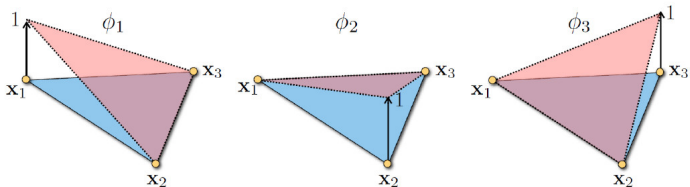
Consider the piece-wise linear coordinate function

$$v(u) = \sum_{i=1}^n \phi_i(u) v_i$$

defining a vector field over the mesh. This vector field can be seen as a smooth generalization of the mesh meaning that

$$\forall u_j \in \Omega: v(u_j) = v_j \quad \Rightarrow \quad \phi_i(u_j) = \delta_{ij}$$

This implies that the basis functions ϕ are hat functions and fulfill the above condition.





The gradient of this vector field is given by

$$\nabla \mathbf{v}(\mathbf{u}) = \sum_{i=1}^n \nabla \phi_i(\mathbf{u}) \mathbf{v}_i^T$$

where the gradient of each basis function is constant per triangle. For a triangle with indices i, j, k , we can obtain them by encoding their properties into an equation system:

$$\begin{pmatrix} (\mathbf{v}_i - \mathbf{v}_k)^T \\ (\mathbf{v}_j - \mathbf{v}_k)^T \\ \mathbf{n}^T \end{pmatrix} \begin{pmatrix} \nabla \phi_i & \nabla \phi_j & \nabla \phi_k \end{pmatrix} = \begin{pmatrix} 1 & 0 & -1 \\ 0 & 1 & -1 \\ 0 & 0 & 0 \end{pmatrix}$$

- Inverse magnitude equal to the length h

$$\|\nabla \phi_i\| = \frac{1}{h} \Leftrightarrow \langle \nabla \phi_i | \mathbf{v}_i - \mathbf{v}_k \rangle = 1$$

- Perpendicular to the opposite edge $\mathbf{v}_j - \mathbf{v}_k$

$$\langle \nabla \phi_i | \mathbf{v}_j - \mathbf{v}_k \rangle = 0$$

- Perpendicular to the triangle normal $\mathbf{n}$

$$\langle \nabla \phi_i | \mathbf{n} \rangle = 0$$



Therefore the vector field gradient is also constant per triangle F_j .

$$\mathbf{g}_j := \nabla \mathbf{v}|_{F_j}$$

So we can build an operator $\mathbf{G} \in \mathbb{R}^{3|F| \times |V|}$ that computes the gradient of the coordinate function

$$\begin{pmatrix} \mathbf{g}_1 \\ \vdots \\ \mathbf{g}_{|F|} \end{pmatrix} = \mathbf{G} \begin{pmatrix} \mathbf{v}_1 \\ \vdots \\ \mathbf{v}_{|V|} \end{pmatrix}$$

and then manipulate the gradients to reflect our deformation:

$$\mathbf{g}_j \mapsto \mathbf{T}_j(\mathbf{g}_j)$$

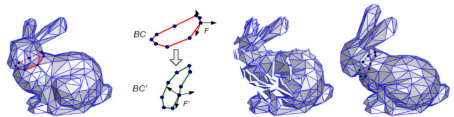
Finally, the new coordinates are obtained by solving the Euler-Lagrange equation

$$\underbrace{\mathbf{G}^T \mathbf{D} \mathbf{G}}_{\Delta} \begin{pmatrix} \mathbf{v}'_1 \\ \vdots \\ \mathbf{v}'_{|V|} \end{pmatrix} = \underbrace{\mathbf{G}^T \mathbf{D}}_{\text{div}} \begin{pmatrix} \mathbf{T}_j(\mathbf{g}_1) \\ \vdots \\ \mathbf{T}_j(\mathbf{g}_{|F|}) \end{pmatrix}$$

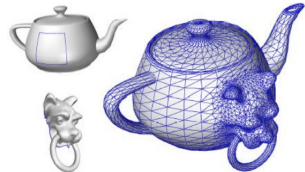
where $\mathbf{D}$ is a diagonal matrix containing the triangle areas.



Mesh Editing: Modify the gradients of the source mesh. Use the old mesh topology and the new gradient values to get a new set of vertex positions.



Mesh Merging: Link the two topologies together. Propagate the changes in gradient information at the boundaries.

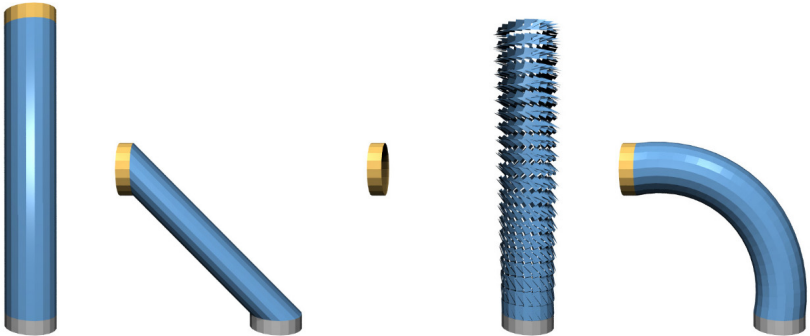


Mesh Smoothing: Smooth the mesh normals. Alter the gradients to reflect the changed normals, then solve for new vertex positions.





Problem: The resulting deformation is not intuitive.



The approach tries to preserve **the original mesh gradients with their orientation** in the global coordinate system.

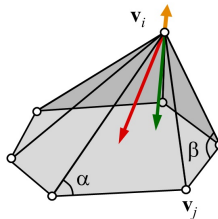
This ignores the fact that in the deformed surface the gradients should rotate as a consequence of the transformation of the vertices and thus the triangles.



Idea: Instead of manipulating per-face gradients, try to deform per-vertex Laplacians.

$$\delta(\mathbf{u}) \mapsto \mathbf{T}(\delta(\mathbf{u})) \quad , \quad \delta(\mathbf{u}) = \Delta f(\mathbf{u})$$

In case of meshes, the delta coordinates are direction vectors pointing into the negative normal direction.



Uniform Laplacian $L_u(v_i)$
Cotangent Laplacian $L_c(v_i)$
Mean curvature normal

For nearly equal edge lengths
Uniform $\approx$ Cotangent

Find a new function f' whose Laplacian field best matches the target:

$$E[f'] = \int_{\Omega} \left\| \Delta f'(\mathbf{u}) - \mathbf{T}(\delta(\mathbf{u})) \right\|_2^2 d\mathbf{u}$$

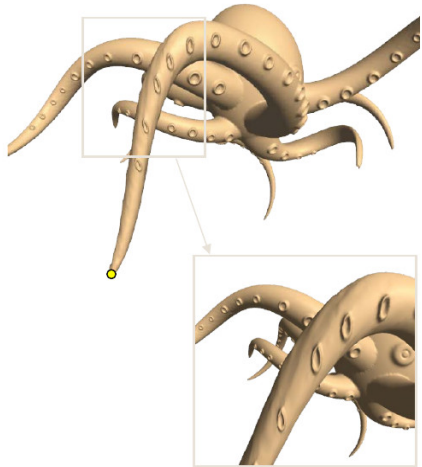
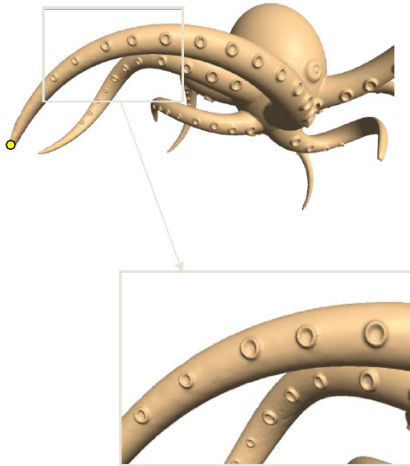
Applying variational calculus leads to the following Euler-Lagrange equation:

$$\Delta^2 f'(\mathbf{u}) = \Delta \mathbf{T}(\delta(\mathbf{u}))$$

Compared to the gradient-based method, Laplacian-based editing leads to C^1 -continuous instead of only C^0 -continuous functions f' .



Problem: The reconstruction still attempts to preserve **the original global orientation** of the details





Definition

Given an $n \times n$ matrix $A = (a_{ij})$ over real or complex numbers, we define the exponential e^A of A as:

$$e^A = I_n + \sum_{p \geq 1} \frac{A^p}{p!} = \sum_{p \geq 0} \frac{A^p}{p!},$$

where $A^0 = I_n$.

The main question is: Why is this definition well-defined?

Theorem

Let $A = (a_{ij})$ be a $n \times n$ matrix. Define $\mu = \max\{|a_{ij}| \mid 1 \leq i, j \leq n\}$. If $A^p = (a_{ij}^{(p)})$, then:

$$|a_{ij}^{(p)}| \leq (n\mu)^p \text{ for all } i, j, 1 \leq i, j \leq n.$$

As a consequence of the proposition we get

- ▶ The n^2 series $\sum_{p \geq 0} \frac{a_{ij}^{(p)}}{p!}$ converge absolutely.
- ▶ The matrix $e^A = \sum_{p \geq 0} \frac{A^p}{p!}$ is a well-defined matrix.

Example: the exponential of a real skew-symmetric matrix:

$$A = \begin{pmatrix} 0 & -\theta \\ \theta & 0 \end{pmatrix}$$

First, observe that:

$$A = \theta \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} \quad \text{and} \quad A^2 = -\theta^2 \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

Letting $J = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$, we derive the following:

$$\begin{aligned} A^{4n} &= \theta^{4n} I_2, \\ A^{4n+1} &= \theta^{4n+1} J, \\ A^{4n+2} &= -\theta^{4n+2} I_2, \\ A^{4n+3} &= -\theta^{4n+3} J. \end{aligned}$$

Using the above inductive formula, the exponential of A is given by:

$$e^A = I_2 + \frac{\theta}{1!} J - \frac{\theta^2}{2!} I_2 - \frac{\theta^3}{3!} J + \frac{\theta^4}{4!} I_2 + \frac{\theta^5}{5!} J - \frac{\theta^6}{6!} I_2 - \frac{\theta^7}{7!} J + \dots$$

Rearranging the terms in the series for e^A :

$$\begin{aligned} e^A &= \left(1 - \frac{\theta^2}{2!} + \frac{\theta^4}{4!} - \frac{\theta^6}{6!} + \cdots \right) I_2 \\ &\quad + \left(\frac{\theta}{1!} - \frac{\theta^3}{3!} + \frac{\theta^5}{5!} - \frac{\theta^7}{7!} + \cdots \right) J. \end{aligned}$$

The series can be recognized as the power series for cosine and sine functions:

$$\begin{aligned} e^A &= \cos(\theta)I_2 + \sin(\theta)J \\ &= \begin{pmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{pmatrix}. \end{aligned}$$

This shows that in this example e^A is a rotation matrix.

Note: This is a general fact. If A is a skew symmetric matrix, then e^A is an orthogonal matrix of determinant $+1$, i.e., a rotation matrix. Furthermore, every rotation matrix is of this form; i.e., the exponential map from the set of skew symmetric matrices to the set of rotation matrices is surjective.



General Linear Group

The set of real invertible $n \times n$ matrices forms a group under multiplication, denoted by $GL(n, \mathbb{R})$.

Special Linear Group

The subset of $GL(n, \mathbb{R})$ consisting of those matrices having determinant $+1$ is a subgroup of $GL(n, \mathbb{R})$, denoted by $SL(n, \mathbb{R})$.

Orthogonal and Special Orthogonal Groups

The set of real $n \times n$ orthogonal matrices forms a group under multiplication, denoted by $O(n)$. The subset of $O(n)$ consisting of those matrices having determinant $+1$ is a subgroup of $O(n)$, denoted by $SO(n)$. Matrices in $SO(n)$ are also called rotation matrices.

Vector Spaces

The set of real $n \times n$ matrices with null trace forms a vector space under addition, as does the set of skew-symmetric matrices.



Definition

The group $GL(n, \mathbb{R})$ is called the general linear group, and its subgroup $SL(n, \mathbb{R})$ is called the special linear group. The group $O(n)$ of orthogonal matrices is called the orthogonal group, and its subgroup $SO(n)$ is called the special orthogonal group (or group of rotations). The vector space of real $n \times n$ matrices with null trace is denoted by $\mathfrak{sl}(n, \mathbb{R})$, and the vector space of real $n \times n$ skew-symmetric matrices is denoted by $\mathfrak{so}(n)$.

Theorem

*The exponential map $\exp : \mathfrak{so}(n) \rightarrow SO(n)$ is well-defined and surjective.*⁵

⁵J. Q. Gallier and J. Quaintance. *Differential geometry and lie groups*. Vol. 12. Springer, 2020.



Consider a 3x3 real skew-symmetric matrix A^6 :

$$A = \begin{pmatrix} 0 & -c & b \\ c & 0 & -a \\ -b & a & 0 \end{pmatrix}$$

Let $\theta = \sqrt{a^2 + b^2 + c^2}$ and define matrix B as:

$$B = \begin{pmatrix} a^2 & ab & ac \\ ab & b^2 & bc \\ ac & bc & c^2 \end{pmatrix}$$

Theorem

(Rodrigues's Formula, 1840): The exponential map $\exp : \mathfrak{so}(3) \rightarrow SO(3)$ is given by:

$$e^A = \cos(\theta)I_3 + \frac{\sin(\theta)}{\theta}A + \frac{1 - \cos(\theta)}{\theta^2}B$$

Or, equivalently, by:

$$e^A = I_3 + \frac{\sin(\theta)}{\theta}A + \frac{1 - \cos(\theta)}{\theta^2}A^2$$

if $\theta \neq 0$, with $e^{0_3} = I_3$.

⁶ L. O. Gallier and J. Quaintance, *Differential geometry and lie groups*, Vol. 12, Springer, 2020.

Sketch of Proof of Rodrigues's Formula

First observe that $A^2 = -\theta^2 I_3 + B$, since

$$\begin{aligned} A^2 &= \begin{pmatrix} 0 & -c & b \\ c & 0 & -a \\ -b & a & 0 \end{pmatrix} \begin{pmatrix} 0 & -c & b \\ c & 0 & -a \\ -b & a & 0 \end{pmatrix} = \begin{pmatrix} -c^2 - b^2 & ba & ca \\ ab & -c^2 - a^2 & cb \\ ac & cb & -b^2 - a^2 \end{pmatrix} \\ &= \begin{pmatrix} -a^2 - b^2 - c^2 & 0 & 0 \\ 0 & -a^2 - b^2 - c^2 & 0 \\ 0 & 0 & -a^2 - b^2 - c^2 \end{pmatrix} + \begin{pmatrix} a^2 & ba & ca \\ ab & b^2 & cb \\ ac & cb & c^2 \end{pmatrix} \\ &= -\theta^2 I_3 + B, \end{aligned}$$

and that $AB = BA = 0$.

From the above, deduce that $A^3 = -\theta^2 A$, and for any $k \geq 0$,

$$A^{4k+1} = \theta^{4k} A, \quad A^{4k+2} = \theta^{4k} A^2, \quad A^{4k+3} = -\theta^{4k+2} A, \quad A^{4k+4} = -\theta^{4k+2} A^2.$$

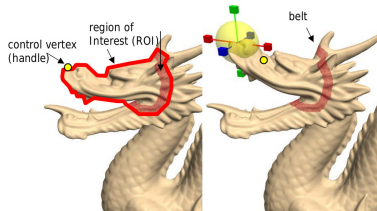
Then prove the desired result by writing the power series for e^A and regrouping terms so that the power series for $\cos(\theta)$ and $\sin(\theta)$ show up. In particular,

$$\begin{aligned}
 e^A &= I_3 + \sum_{p \geq 1} \frac{A^p}{p!} \\
 &= I_3 + \sum_{p \geq 0} \frac{A^{2p+1}}{(2p+1)!} + \sum_{p \geq 1} \frac{A^{2p}}{(2p)!} \\
 &= I_3 + \sum_{p \geq 0} \frac{(-1)^p \theta^{2p}}{(2p+1)!} A - \sum_{p \geq 1} \frac{(-1)^{p-1} \theta^{2(p-1)}}{(2p)!} A^2 \\
 &= I_3 + \frac{\sin(\theta)}{\theta} A + (1 - \cos(\theta)) \frac{A^2}{\theta^2}.
 \end{aligned}$$

The above formulae are the well-known formulae expressing a rotation of axis specified by the vector (a, b, c) and angle θ . Since the exponential is surjective, it is possible to write down an explicit formula for its inverse (but it is a multivalued function!).



Observation: Gradient- and Laplacian-based Editing assumes that deformation of the whole gradient and Laplacian field is known. However, the user only wants to modify a specific region (ROI).



Idea: Solve for both the deformed mesh V' and the local transformations T_i ⁷:

$$E[V'] = \sum_{i=1}^n \left\| \Delta v'_i - T_i(\delta_i) \right\|_2^2$$

Since the transformations T_i depend on the new mesh V' , they are implicitly optimized.

⁷O. Sorkine, D. Cohen-Or, et al. "Laplacian surface editing". In: *Proceedings of the 2004 Eurographics/ACM SIGGRAPH symposium on Geometry processing*. ACM. 2004, pp. 175–184.



Each transformation T_i should map the vertex v_i and its neighbors to the deformed unknown vertices:

$$\left\| T_i v_i - v'_i \right\|_2^2 + \sum_{v_j \in \mathcal{N}(v_i)} \left\| T_i v_j - v'_j \right\|_2^2$$

To avoid undesired solutions when minimizing this energy, we further constrain the transformations (in homogeneous coordinates) such that they satisfy the following conditions:

- ▶ Isotropic Scales $s_i \in \mathbb{R}$
- ▶ Rotations $\omega_i \in \mathbb{R}^3$
- ▶ Translations $t_i \in \mathbb{R}^3$

$$T_i = \begin{pmatrix} s_i \exp[\omega_i]_{\times} & t_i \\ \mathbf{0} & 1 \end{pmatrix} \in \mathbb{R}^{4 \times 4}$$

Rotation matrices $R_i \in \mathbb{R}^{3 \times 3}$ have 3 degrees of freedom (DoF) and can be efficiently represented as 3D vectors ω_i .

$$[\omega_i]_{\times} = \begin{pmatrix} 0 & -\omega_{i,z} & \omega_{i,y} \\ \omega_{i,z} & 0 & -\omega_{i,x} \\ -\omega_{i,y} & \omega_{i,x} & 0 \end{pmatrix}, \quad \forall x \in \mathbb{R}^3: [\omega_i]_{\times} x = \omega_i \times x$$

Here, $[\omega_i]_{\times}$ denotes the cross-product matrix.



Assuming small rotation angles $\|\omega_i\|_2$, we can linearize the rotation to obtain

$$s_i \exp[\omega_i]_{\times} \approx s_i (\mathbf{I} + [\omega_i]_{\times}) = s_i \mathbf{I} + [\mathbf{h}_i]_{\times} \quad , \quad \mathbf{h}_i = s_i \cdot \omega_i$$

The scale factor s_i is moved into the rotation vector ω_i to make the matrix linear in the unknown parameters:

$$\mathbf{T}_i = \begin{pmatrix} s_i & -h_{i,z} & h_{i,y} & t_{i,x} \\ h_{i,z} & s_i & -h_{i,x} & t_{i,y} \\ -h_{i,y} & h_{i,x} & s_i & t_{i,z} \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Finally, we plug this into the energy function

$$\left\| \mathbf{T}_i \mathbf{v}_i - \mathbf{v}'_i \right\|_2^2 + \sum_{\mathbf{v}_j \in \mathcal{N}(\mathbf{v}_i)} \left\| \mathbf{T}_i \mathbf{v}_j - \mathbf{v}'_j \right\|_2^2$$

and rewrite it to a linear least squares problem.



Therefore, we can write our problem as

$$\left\| A_i \begin{pmatrix} s_i \\ \mathbf{h}_i \\ \mathbf{t}_i \end{pmatrix} - \mathbf{b}_i \right\|_2^2$$

where

$$A_i = \begin{pmatrix} \vdots \\ \mathbf{v}_k & [\mathbf{v}_k]_{\times} & \mathbf{I} \\ \vdots \end{pmatrix} \in \mathbb{R}^{3|\mathcal{I}| \times 7}, \quad \mathbf{b}_i = \begin{pmatrix} \vdots \\ \mathbf{v}'_k \\ \vdots \end{pmatrix} \in \mathbb{R}^{3|\mathcal{I}|}$$

and $k \in \mathcal{I} = \{i\} \cup \{j \mid \mathbf{v}_j \in \mathcal{N}(\mathbf{v}_i)\}$. Here, A_i only depends on the given vertices $\mathbf{v}_i$ whereas $\mathbf{b}_i$ contains the unknown vertices $\mathbf{v}'_i$. Therefore, the solution

$$\begin{pmatrix} s_i \\ \mathbf{h}_i \\ \mathbf{t}_i \end{pmatrix} = \underbrace{(A_i^T A_i)^{-1} A_i}_{=: X_i} \mathbf{b}_i$$

is obtained by applying the linear function X_i to the unknown vertices.



The transformations T_i are not explicitly computed. Instead, we plug its linear approximation and solution into the original energy function

$$E[\mathbf{V}'] = \sum_{i=1}^n \left\| \Delta \mathbf{v}_i' - \mathbf{T}_i(\delta_i) \right\|_2^2$$

to obtain

$$E[\mathbf{V}'] = \sum_{i=1}^n \left\| \Delta \mathbf{v}_i' - \mathbf{Y}_i(\delta_i) \mathbf{v}_i' \right\|_2^2$$

where $\mathbf{Y}_i(\delta_i)$ is the matrix that encodes the transformation and the delta coordinates. The fixed points $\mathbf{u}_l$ are added as a soft constraint into the energy function:

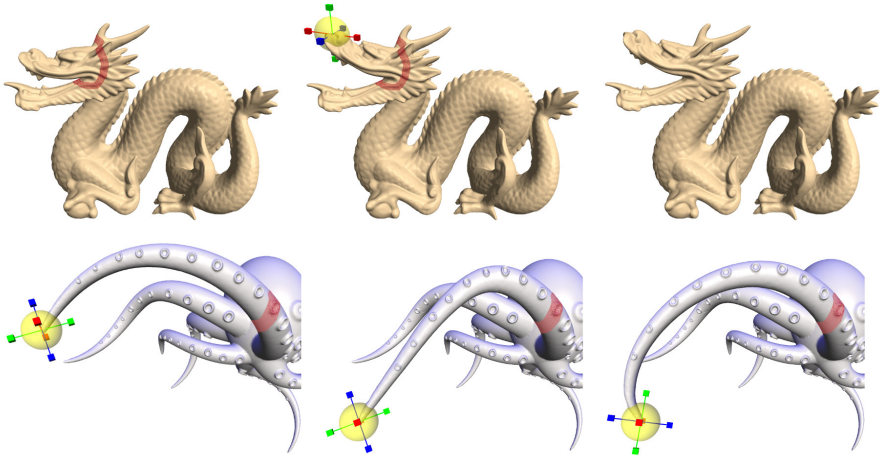
$$E[\mathbf{V}'] = \sum_{i=1}^n \left\| \Delta \mathbf{v}_i' - \mathbf{Y}_i(\delta_i) \mathbf{v}_i' \right\|_2^2 + \sum_{l \in \mathcal{C}} \left\| \mathbf{v}_l' - \mathbf{u}_l \right\|_2^2$$

Note: In case of too large rotation angles, an approximated reconstruction⁸ can be computed and then refined using this technique.

⁸Y. Lipman et al. "Differential coordinates for interactive mesh editing". In: *Shape Modeling Applications*, 2004. *Proceedings*. IEEE. 2004, pp. 181–190.



Results:





- [1] M. Botsch, R. Sumner, M. Pauly, and M. Gross. “Deformation Transfer for Detail-Preserving Surface Editing”. In: *Vision, Modeling & Visualization*. 2006.
- [2] J. Q. Gallier and J. Quaintance. *Differential geometry and lie groups*. Vol. 12. Springer, 2020.
- [3] Y. Lipman, O. Sorkine, D. Cohen-Or, D. Levin, C. Rossi, and H.-P. Seidel. “Differential coordinates for interactive mesh editing”. In: *Shape Modeling Applications, 2004. Proceedings*. IEEE. 2004, pp. 181–190.
- [4] O. Sorkine and M. Botsch. “Interactive Shape Modeling and Deformation”. In: *Eurographics 2009 - Tutorials*. Ed. by K. Museth and D. Weiskopf. The Eurographics Association, 2009.
- [5] O. Sorkine, D. Cohen-Or, Y. Lipman, M. Alexa, C. Rössl, and H.-P. Seidel. “Laplacian surface editing”. In: *Proceedings of the 2004 Eurographics/ACM SIGGRAPH symposium on Geometry processing*. ACM. 2004, pp. 175–184.
- [6] D. Terzopoulos, J. Platt, A. Barr, and K. Fleischer. “Elastically deformable models”. In: *ACM Siggraph Computer Graphics*. Vol. 21. 4. ACM. 1987, pp. 205–214.



- [7] Y. Yu, K. Zhou, D. Xu, X. Shi, H. Bao, B. Guo, and H.-Y. Shum. “Mesh editing with poisson-based gradient field manipulation”. In: *ACM Transactions on Graphics (TOG)*. Vol. 23. 3. ACM. 2004, pp. 644–651.